

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ОТЧЕТ
по лабораторной работе №6
на тему
«Работа с файлами, отображенными в память»

Выполнил:
студ. гр. 450502 Бурш А.П.
Проверил:
пр. Благиных В.Д.

Минск, 2026

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Написать многопоточную программу `sort_index` для сортировки вторичного индексного файла таблицы базы данных, работающую с файлом с использованием отображение файлов в адресное пространство процесса. Программа должна запускаться следующим образом:

```
$ sort_index memsize blocks threads filename
```

Параметры командной строки:

`memsize` – размер рабочего буфера, кратный размеру страницы (`getpagesize(2)`);
`blocks` – порядок (количество блоков) разбиения буфера;
`threads` – количество потоков (от k до N), где k – количество процессорных ядер, N – максимальное количество потоков ($k \leq N \leq 8k$);
`filename` – имя файла.

Количество блоков должно быть степенью двойки и превышать количество потоков не менее, чем в 4 раза (при $k = 4$ количество блоков не должно быть менее 16).

Для целей тестирования следует написать программу `gen`, которая будет генерировать неотсортированный индексный файл, и программу `view` для отображения индексного файла на `stdout`.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Описание алгоритма выполнения работы

Программа состоит из основного процесса, который в зависимости от выбора пользователя генерирует бинарный файл указанного размера с неотсортированными индексными записями, выводит содержимое этого файла или сортирует его. Программа обрабатывает ошибки и корректно освобождает ресурсы.

2.2 Описание алгоритма программы генерации

Генерируемый файл представляет собой вторичный индекс по времени и состоит из заголовка и индексных записей фиксированной длины.

Индексная запись имеет следующую структуру:

```
struct index_s {  
    double time_mark;  
    uint64_t recno;  
};
```

Заголовок представляет собой следующую структуру:

```

struct index_hdr_s {
    uint64_t records;    // количество записей
    struct index_s idx[]; // массив записей в количестве records
};

```

Временная метка определяется в модифицированный юлианских днях. Целая часть лежит в пределах от 15020.0 (1900.01.01-0:0:0.0) до «вчера». Дробная – это часть дня (0.5 – 12:0:0.0). Для генерации целой и дробной частей временной метки следует использовать системный генератор случайных чисел `rand(3)` или `rand_r(3)`. Первичный индекс, как вариант, может заполняться последовательно, начиная с 1, но может быть случайным целым > 0 (в программе сортировки не используется).

Размер индекса в записях должен быть кратен 256 и кратно превышать планируемую выделенную память для отображения. Размер индекса и имя файла указывается при запуске программы генерации.

2.3 Описание алгоритма программы сортировки

1) Основной поток запускает `threads` потоков, сообщая им адрес буфера, размер блока `memsize/blocks`, и их номер от 1 до `threads - 1`, используя возможность передачи аргумента для `start_routine`. Порожденные потоки останавливаются на барьере, ожидая прихода основного.

2) Основной поток с номером 0 открывает файл, отображает его часть размером `memsize` на память и синхронизируется на барьере. Барьер «открывается» и все `threads` потоков входят на равных в фазу сортировки.

3) Фаза сортировки

С каждым из блоков связана карта (массив) отсортированных блоков, в которой изначально блоки с 0 по `threads-1` отмечены, как занятые.

Поток `n` начинает с того, что выбирает из массива блок со своим номером и его сортирует, используя `qsort(3)`. После того, как поток отсортировал свой первый блок, он на основе конкурентного захвата мьютекса, связанного с картой, получает к ней эксклюзивный доступ, отмечает следующий свободный блок, как занятый, освобождает мьютекс и приступает к его сортировке.

Если свободных блоков нет, синхронизируется на барьере. После прохождения барьера все блоки будут отсортированы.

4) Фаза слияния

Поскольку блоков степень двойки, слияния производятся парами в цикле.

Поток 0 сливает блоки 0 и 1, поток 1 – блоки 2 и 3, и так далее.

Для отметки слитых пар и не слитых используется половина карты. Если для потока нет пары слияния, он синхронизируется на барьере.

В результате слияния количество блоков, подлежащих слиянию сокращается в два раза, а размер их в два раза увеличивается.

После очередного прохождения барьера количество блоков, подлежащих слиянию, станет меньше количества потоков. В этом случае распределение блоков между потоками осуществляется на основе конкурентного захвата мьютекса, связанного с картой. Потоки, которым не досталось блока, синхронизируются на барьере.

Когда осталась последняя пара, все потоки с номером не равным нулю синхронизируются на барьере, а поток с номером 0 выполняет слияние последней пары.

После слияния буфер становится отсортирован и подлежит сбросу в файл (munmap()).

Если не весь файл обработан, продолжаем с шага 2.

Если весь файл обработан, основной поток отправляет запрос отмены порожденным потокам, выполняет слияние отсортированных частей файла и завершается.

2.4 Описание алгоритма программы вывода содержимого файла

Программа открывает бинарный файл с указанным именем в режиме чтения, а затем входит в цикл до конца файла, в котором читает оттуда одну индексную запись, выводит её первичный индекс и временную метку.

3 ОПИСАНИЕ ПОРЯДКА СБОРКИ

Проект собирается с помощью системы CMake, которая автоматически настраивает процесс компиляции исходного кода и подготавливает создание исполняемых файлов. В данной конфигурации собираются три отдельные программы: gen, view и sort_index. Все они создаются из исходных файлов, расположенных в каталоге src, а для компиляции используется стандарт языка C11.

4 РЕЗУЛЬТАТЫ РАБОТЫ ПРОГРАММЫ

```
=====
sort_index workspace
=====

Current configuration
file   : TEST
memSize : 4096 bytes
blocks : 32
threads : 8

Actions
[1] Generate index file
[2] View index file
[3] Sort index file
[q] Quit

Select action: 2
Viewing index file: TEST

Row   Record      Time mark
-----
1     1           35033.93663
2     2           57707.21414
3     3           35656.51654
4     4           40553.58253
5     5           28523.27690
6     6           66455.80903
7     7           38387.51536
8     8           48030.71523
9     9           73258.41676
10    10          65118.96913
11    11          45490.17370
12    12          22146.02017
13    13          69841.51526
14    14          43525.61411
15    15          70465.07311
16    16          63126.99117
17    17          74809.37232
18    18          29988.14065
19    19          68333.39183
20    20          24989.40350
21    21          22060.67175
22    22          25134.85926
23    23          37171.23486
24    24          75195.83601
25    25          67276.70063
26    26          72996.50779
27    27          39286.17671
28    28          47289.16451
250   250         20132.57613
251   251         25512.12412
252   252         63104.97751
253   253         75751.65951
254   254         28657.22628
255   255         52677.94328
256   256         32397.71937

131043 Total records: 256
```

Рисунок 1 – Генерация неотсортированного файла с индексными записями и его просмотр

```

Select action: 3

Sorting session
=====
=====
file size : 4096 bytes
memSize   : 4096 bytes
blocks    : 32
threads   : 8
chunks    : 1
target     : TEST
=====
=====
Processing chunk 1/1 (offset 0, size 4096 bytes)
Sorting completed successfully. Output file: TEST
=====
=====
sort_index workspace
=====
=====
Current configuration
file       : TEST
memSize    : 4096 bytes
blocks     : 32
threads    : 8

Actions
[1] Generate index file
[2] View index file
[3] Sort index file
[q] Quit

Select action: 2
Viewing index file: TEST

Row  Record      Time mark
-----
1    107          15601.52005
2    206          15604.91763
3    75           15876.86958
4    80           15969.58758
5    171          16190.57474
250  151           74306.13252
251  17           74809.37232
252  99           74955.12150
253  24           75195.83601
254  181          75275.31061
255  235          75456.82147
256  253          75751.65951

Total records: 256

```

Рисунок 2 – Сортировка, вывод содержимого отсортированного файла и завершение программы

ЗАКЛЮЧЕНИЕ

При выполнении лабораторной работы были разработаны программа `gen` для генерации файла указанного размера, содержащего неотсортированные индексные записи, программа `view` для вывода содержимого файла на `stdout`, а также программа `sort_index` для удобного вызова генерации и вывода содержимого файла с индексными записями, а также его сортировки, используя буфер указанного размера, разделяя файл на указанное при запуске количество блоков, а также используя заданное количество потоков.